

Design and Implementation of an Arduino-Based Autonomous Line-Following Robot with Onboard Distance Measurement

Sujeet Kumar Jha, Saurab Dulal, Manish Karn, and Ahmed Raja Khan

Abstract—Line following is one of the most fundamental tasks in mobile robotics and serves as a stepping stone toward more advanced autonomous navigation systems. This paper presents the design, implementation, and evaluation of an autonomous line-following robot capable of tracking a black line drawn on a white surface using an array of optical (IR-LED/photodiode) sensors. The four-sensor array gives the platform precise and flexible path tracking, while two DC gear motors, driven through an L293D H-bridge under Arduino Uno control, provide differential steering. A proportional duty-cycle control scheme adjusts the pulse-width-modulated (PWM) signal supplied to each motor so that the robot can travel straight, turn, or execute sharp turns in response to the sensor pattern, and it re-acquires the line autonomously by sweeping in a circular search pattern whenever it loses track. An auxiliary infrared break-beam sensor coupled to a wheel is used to build a low-cost digital revolution counter, whose count is converted into a travelled-distance estimate and shown on an onboard liquid-crystal display (LCD). The complete circuit was first verified in Proteus simulation and then implemented on hardware. Practical measurements of motor speed against PWM duty cycle, together with a comparison between the constructed tachometer and a commercial reference instrument (error within approximately 6–12%), are reported. The final electromechanical platform measures $192 \times 100 \times 70 \text{ mm}^3$, weighs approximately 200 g, and achieves a minimum turning radius of 100 mm at a velocity of about 24.2 cm/s. The system demonstrates that a low-cost, interdisciplinary combination of optical sensing, embedded control, and differential-drive kinematics can yield a reliable line-tracking platform suitable for automated-guidance applications such as industrial material carriers, museum tour guides, and household or educational robotics.

Index Terms—Autonomous robot, Arduino, differential steering, DC motor control, line-following robot, optical sensor, pulse-width modulation (PWM), tachometer.

I. INTRODUCTION

A Line follower is a mobile machine that autonomously tracks a path – typically a black line on a white (or otherwise contrasting) surface – by continuously sensing the line and correcting its trajectory in a closed feedback loop. Although conceptually simple, the line-following problem is a useful vehicle for exploring sensing, actuation, and control, and it has practical applications in automobile guidance and industrial automation [1].

The authors are with the Department of Electrical and Electronics Engineering, School of Engineering, Kathmandu University, Dhulikhel, Nepal (ID: sujeet.jha; saurab.dulal; manish.karn; ahmed.khan).

This project report was prepared as a third-year Bachelor of Engineering project under the supervision of Mr. Bikalpa Upadhyaya, Department of Electrical and Electronics Engineering, Kathmandu University, August 2016.

A. Background

As technology assumes an increasingly central role in modern life, it becomes valuable not only to use existing technology but also to understand how to build it. Engineering projects that combine multiple disciplines – mechanical, electrical, electronic, and software – help develop the cross-cutting skills that narrowly scoped, single-discipline projects do not. The work reported here was undertaken with this philosophy: it integrates mechanical design, analog sensing circuitry, embedded programming, and closed-loop control into a single platform. Among the intended benefits of the exercise are:

- building visual intuition for mathematics and physical principles used in the design;
- developing structured, logical problem-solving skills;
- encouraging innovative and creative engineering solutions;
- improving overall problem-solving ability.

In addition to tracking the line, the robot reported in this paper also estimates and visually displays the distance it has travelled.

B. Motivation

The natural world already exhibits line-following-like behaviour – ants follow invisible pheromone trails between a food source and their colony, and vehicles on a road follow lane markings to avoid collisions. Robotics, in large part, is the exercise of translating such naturally occurring behaviours into engineered machines. Potential beneficiaries of line-following technology include:

- automated equipment carriers in industrial settings;
- entertainment and small household applications;
- automated tour guides in museums and similar exhibition spaces;
- reconnaissance-support platforms.

C. Problem Description

In many industrial settings, carts or trucks are used to move intermediate products between manufacturing stages that are located in different buildings or separate sections of a plant. When such transport relies on human drivers, it introduces unreliability and inefficiency that can become the weakest link in an otherwise automated production line. This project explores automating that transport step using a cart that follows

a painted or taped line rather than a fixed rail track, which is both costly to install and inconvenient to reconfigure.

D. Objectives

The specific objectives of the project were to develop a robot that:

- reliably follows a line drawn on a contrasting surface;
- negotiates turns of varying sharpness;
- is largely insensitive to ambient lighting and electrical noise;
- allows calibration of the line-detection (darkness) threshold;
- is designed with scalability as a primary consideration;
- displays the distance it has travelled on an onboard LCD.

E. Methodology

The project followed a standard design-build-test methodology. A literature survey of relevant books, journals, and online resources was first used to scope the design. The control logic was then derived and implemented as an Arduino sketch (Arduino 1.65), and both the logic and the supporting electronic circuit were verified in the Proteus simulation environment before hardware implementation. Once the electronics and firmware were functioning as intended, a stable and flexible mechanical chassis was designed and fabricated, and the complete system was tested and iteratively debugged.

F. Limitations

The realised system is subject to the following constraints:

- the line colour/contrast expected by the hardware cannot be changed in software;
- threshold calibration is sensitive and not straightforward to set optimally;
- very sharp curves are not tracked efficiently and must be avoided;
- the minimum comfortable turning (U-turn) radius is approximately 100 mm;
- the line width must be at least 45 mm so that it is detected by at least three of the four sensors;
- the track must be planar and free of obstacles;
- the differential-steering approach used here does not scale easily to large or non-electric vehicles.

G. Organisation of the Paper

The remainder of the paper is organised as follows. Section II reviews the hardware and software building blocks of the system, including the sensor array, processing unit, and actuation stage. Section III describes the overall design and implementation, including the schematic, control logic, and control-flow diagrams. Section IV presents experimental results for motor speed control and the constructed digital tachometer. Section V analyses the kinematics of the differential-steering mechanism, the mechanical design, and the cost of the platform. Section VI concludes the paper and outlines directions for future work.

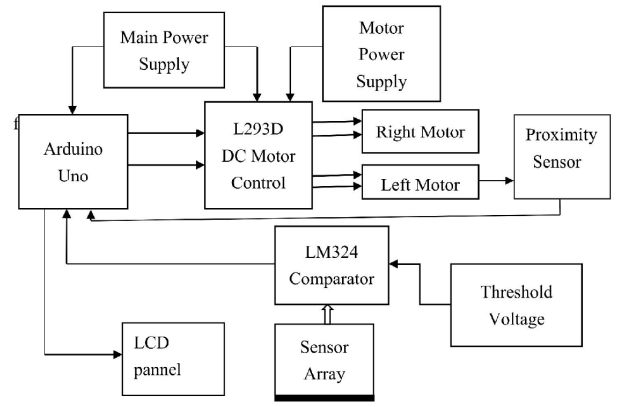


Fig. 1. Block diagram of the line-following robot.

II. SYSTEM OVERVIEW AND LITERATURE SURVEY

A line-following robot is a self-operating platform that detects and follows a track – in this implementation, a black path on a white surface – using optical sensors.

A. Basic Operation

The basic operating principles of the platform are as follows:

- Line position is captured using optical sensors mounted at the front of the robot. Each sensor is a combination of an IR-LED and a photodiode, providing high-resolution, robust line detection.
- Steering is achieved through two independently driven wheels, each controlled by its own motor, forming a differential-drive mechanism.
- An LCD panel displays the distance travelled by the robot.
- If no line is detected, the robot executes a circular search pattern until the line is reacquired.

A block diagram of the overall system is shown in Fig. 1.

B. Input System

1) *Optical Sensors:* The robot senses the line with an array of four photodiode-based optical sensors: an IR-LED (transmitter) and a photodiode (receiver) pair facing the ground. The amount of infrared light reflected back to the photodiode produces an analog signal, which is converted to a binary value (0 or 1) by an analog comparator before being read by the Arduino. The four sensors are arranged in a straight row underneath the front of the chassis, spaced 1.5 cm apart, as shown in Fig. 2. The two left-hand sensors are labelled L1 and L2, and the two right-hand sensors R1 and R2. By convention, a sensor over the line reads 0 and a sensor off the line reads 1; the Arduino uses this 4-bit pattern to decide the next motor command, always seeking a configuration in which all four sensors read 0.

The number of sensors used is a design trade-off: too few sensors makes the tracking response coarse and prone to failure on sharp turns, while too many sensors demands more complex control logic and additional hardware. Four sensors were found to be an adequate compromise for this platform.

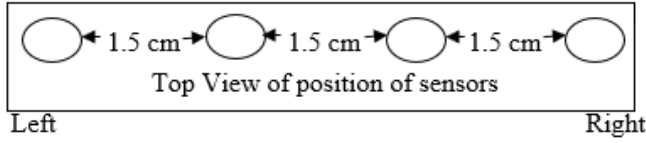


Fig. 2. Arrangement of the optical sensor array (top view).

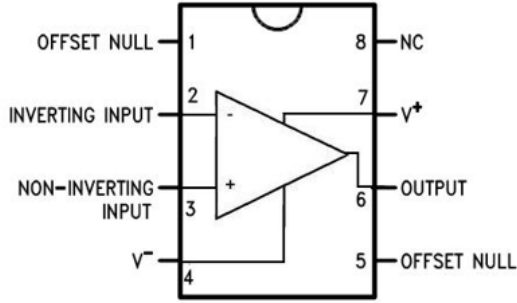
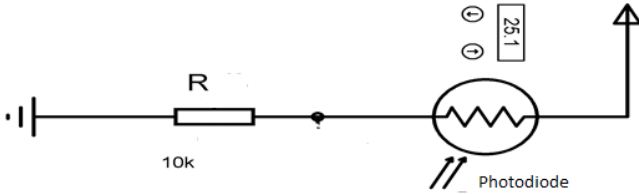


Fig. 3. Comparator logic used to binarise each optical sensor output.

Fig. 4. Optical sensor schematic: photodiode with pull-up resistor $R = 10 \text{ k}\Omega$.

2) *Comparator*: A comparator produces a high or low digital output depending on the relative magnitude of two input voltages, as illustrated in Fig. 3: if the non-inverting input exceeds the inverting input the output is high (logic 1), and otherwise the output is low (logic 0).

For each sensor channel, one comparator input is driven by the photodiode/potential-divider network shown in Fig. 4, while the other is set by a reference (threshold) voltage used to calibrate for the contrast between the line and the surface.

C. Processing Unit

The processing unit is the “brain” of the robot, mapping sensor inputs to the appropriate actuator outputs. This design uses an Arduino Uno board, built around the ATmega328 microcontroller (8-bit CPU, 16 MHz clock, 2 KB SRAM, 32 KB flash, 1 KB EEPROM) [2]. Key features used from the board include 14 digital I/O pins (six of which, pins 3, 5, 6, 9, 10, and 11, support PWM output) and six analog input pins, operating from a 7–12 V supply. The board pinout is shown in Fig. 5.

D. Output System

1) *Motor Driver*: Because the microcontroller cannot source the current required to drive DC motors directly, an L293D dual H-bridge IC is used as a current amplifier and

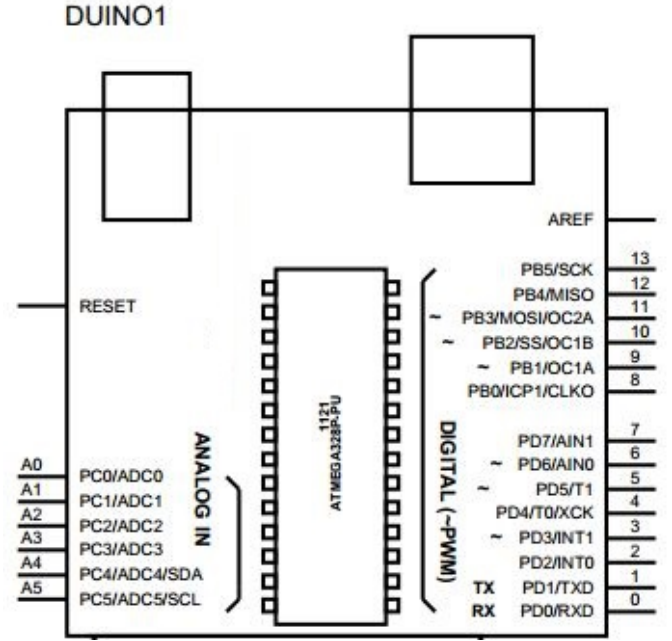


Fig. 5. Arduino Uno pin configuration.

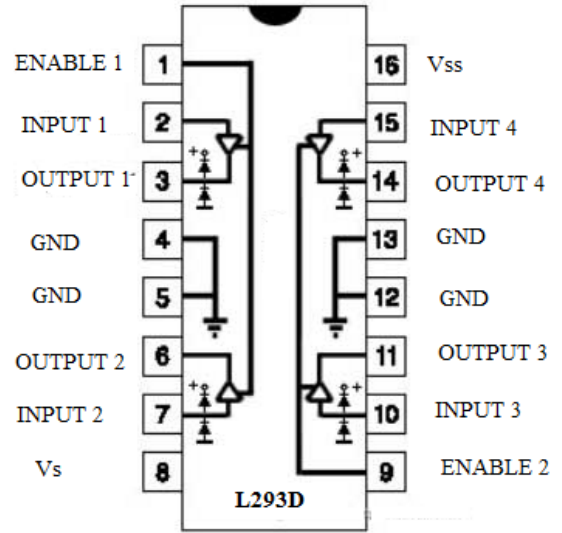


Fig. 6. Pin configuration of the L293D dual H-bridge motor-driver IC.

switching stage between the Arduino and the two drive motors. The L293D can drive two motors simultaneously (one per channel) and accepts a logic-level PWM signal on its ENABLE pins for speed control. Its supply voltage V_{ss} (typically 6 V for a plain DC motor or 6–12 V for a gear motor) is chosen according to motor rating, while its logic supply V_{cc} (nominally +5 V) defines input thresholds of approximately -0.3 – 1.5 V for a logic low and 2.3 – 5 V for a logic high. The pin configuration is shown in Fig. 6.

2) *DC Gear Motor*: Two DC gear motors provide the drive for the robot (Fig. 7). A DC motor is the simplest actuator to control: reversing the polarity of its supply reverses its direction of rotation, and its speed can be varied by varying the effective voltage across it. Because a bare DC motor does

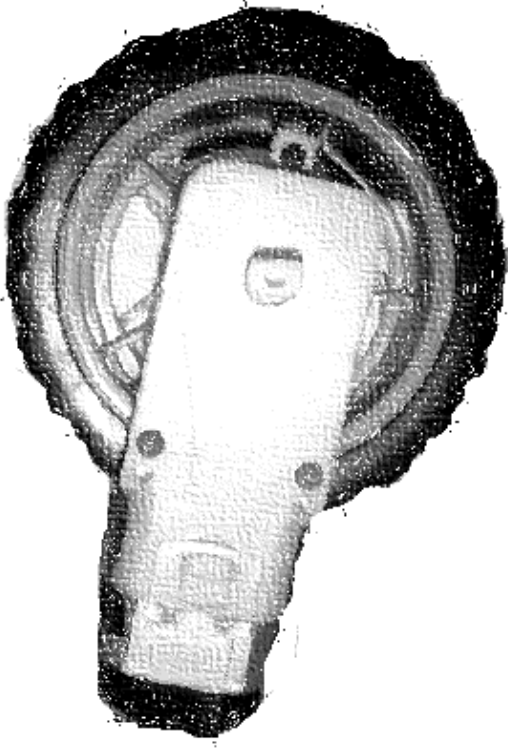


Fig. 7. Low-voltage DC gear motor with attached wheel.

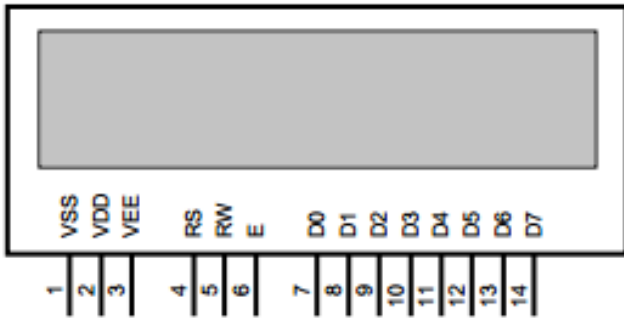


Fig. 8. Schematic of the character LCD panel and its pinout.

not provide sufficient torque to move the chassis directly, a gearbox is used to trade rotational speed for torque.

3) *LCD Display*: A character LCD panel (Fig. 8) displays the distance travelled by the robot, computed from the pulse count produced by the onboard tachometer system.

4) *Proximity Sensor (Tachometer Pickup)*: An additional IR-LED/photodiode pair, arranged as a reflective break-beam sensor (Fig. 9), is used to build a low-cost revolution counter. A low-value resistor drives the IR-LED brightly enough that the photodiode reliably switches “on” whenever the emitted beam is reflected back to it (for example, by a marked spot on the wheel). Each detected pulse is passed to the Arduino as a hardware interrupt, and the resulting count is used, together with the wheel geometry, to compute and display the covered distance on the LCD.

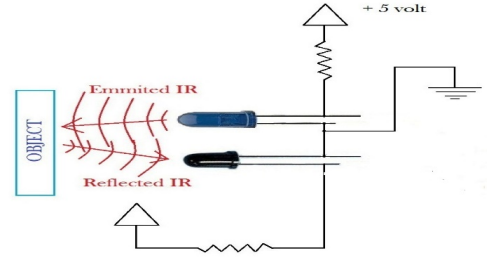


Fig. 9. Reflective IR proximity sensor used as the tachometer pickup.

E. Software Tools

The circuit and control logic were verified in simulation using Proteus®, and the firmware was written and uploaded to the Arduino Uno using the Arduino 1.65 IDE.

F. Section Summary

The complete system decomposes cleanly into three subsystems. The *input system* comprises an array of four IR-LED/photodiode pairs arranged in a straight line; each sensor’s output is compared against a calibrated threshold voltage using an analog comparator, and the four resulting binary signals are fed to the Arduino’s digital inputs [3]. The *processing system* is the Arduino Uno, built around the ATmega328 microcontroller, which executes the control logic burned into it. The *output system* combines the L293D driver with the two gear motors to realise six qualitative modes of motion – turn left/right, move left/right, and drift left/right – by varying the relative speed of the two drive wheels.

III. DESIGN AND IMPLEMENTATION

A. Schematic

The complete schematic of the line-following robot, drawn in Proteus, is shown in Fig. 10. The principal hardware building blocks are: the Arduino Uno; four IR-LED/photodiode pairs configured as reflective line sensors; an LM324 quad comparator IC; a potentiometer used to calibrate the reference (threshold) voltage; an L293D H-bridge motor driver; two DC gear motors; and a further IR-LED/photodiode pair used as the tachometer pickup. Each subsystem was designed and verified independently before integration, which considerably simplified debugging.

B. Control Logic

The microcontroller reads four binary sensor inputs (A0–A3) and, based on their combined pattern, selects one of eight qualitative responses, summarised in Table I [4]. Each response is realised by writing an appropriate PWM value (0–255) to the two motor-enable pins (digital pins 9 and 10).

C. Process Explanation

Fig. 11 illustrates the qualitative relationship between the sensor pattern and the resulting robot motion at each control cycle, summarising the mapping realised in software [5].

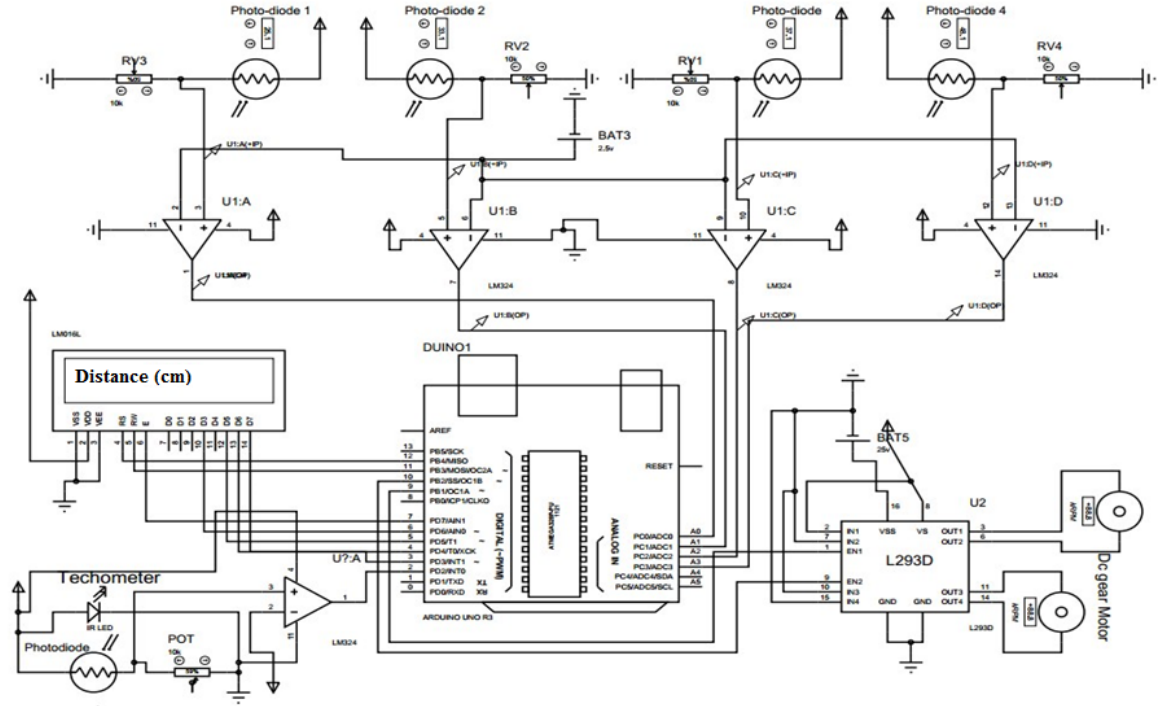


Fig. 10. Complete circuit diagram of the line-following robot, drawn in Proteus.

TABLE I
ARDUINO CONTROL LOGIC

Input				PWM Output		Sensor State	Action
A0	A1	A2	A3	Pin 9	Pin 10		
0	0	0	0	255	255	All sensors on line	Go straight
1	0	0	0	255	191	Leftmost sensor off track	Move right
1	1	0	0	255	127	Two left sensors off track	Turn right
1	1	1	0	255	64	Three left sensors off track	Sharp turn right
0	0	0	1	191	255	Rightmost sensor off track	Move left
0	0	1	1	127	255	Two right sensors off track	Turn left
0	1	1	1	64	255	Three right sensors off track	Sharp turn left
1	1	1	1	0	255	All sensors off track	Circular search until line found

D. Programming and Simulation

The firmware acts as the decision-maker embedded in the microcontroller, mapping each sensor pattern to the corresponding motor command. The sketch was written in the Arduino 1.65 IDE, compiled to a .hex file, and burned onto the Arduino; its behaviour was cross-checked against the Proteus simulation before hardware testing [6]. The complete control-flow logic, including the periodic distance-update routine driven by the tachometer interrupt, is shown in Fig. 12 and Fig. 13.

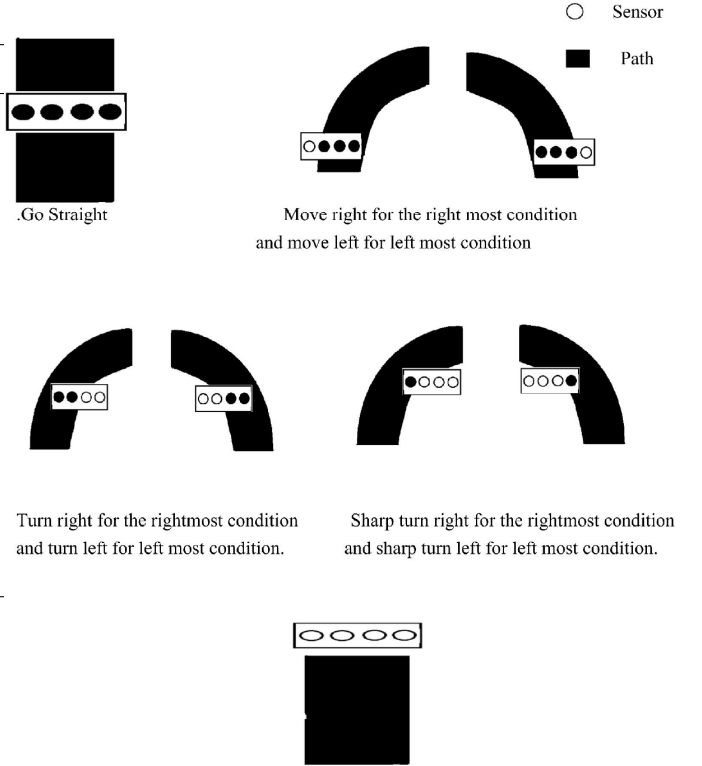


Fig. 11. Line-following control response for representative sensor patterns.

E. Section Summary

The Arduino continuously reads the sensor array, applies the logic of Table I, and writes the corresponding PWM values to

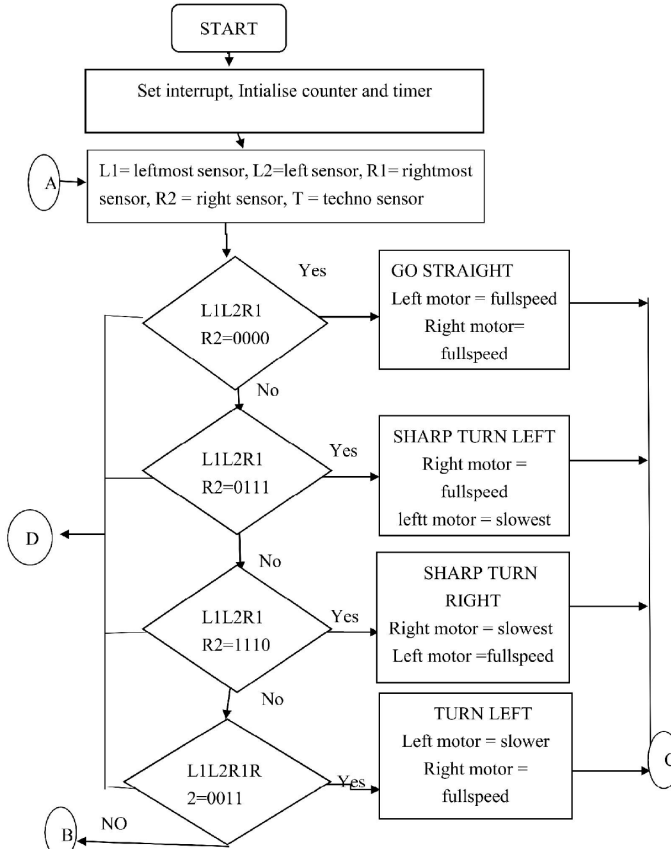


Fig. 12. Control-flow diagram of the line-following algorithm (part 1 of 2).

the motor driver, which in turn drives the two wheel motors to produce the required motion. Because each rear wheel is powered by a dedicated motor while the front wheels are free-rolling, the platform uses a differential-steering system: driving both motors at equal speed produces straight-line motion, while driving the motor on the side of the desired turn at a reduced voltage produces a turn whose sharpness scales with the speed difference between the two wheels.

IV. RESULTS AND ANALYSIS

Practical measurements were carried out to characterise both the motor-speed-control stage and the constructed tachometer.

For a DC motor, rotational power P_r is the product of torque T and rotational speed ω :

$$T = \frac{P_r}{\omega}. \quad (1)$$

Because P_r is approximately constant for a given electrical input power, torque is inversely proportional to speed:

$$T \propto \frac{1}{\omega}. \quad (2)$$

This inverse relationship is the reason a reduction gearbox is used: it trades some of the motor's free-running speed for the additional torque needed to move the chassis.

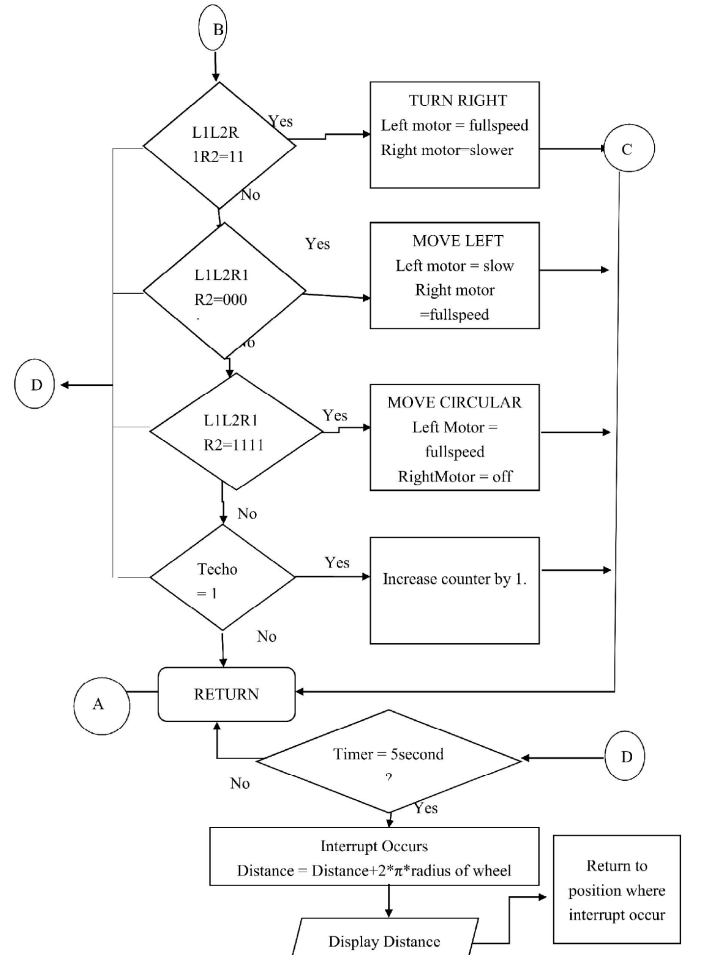


Fig. 13. Control-flow diagram of the line-following algorithm and periodic distance calculation (part 2 of 2).

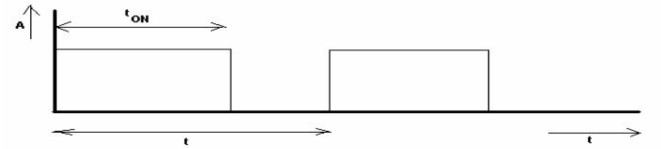


Fig. 14. PWM waveform used for motor speed control.

A. Speed Control

Motor speed is controlled by feeding a PWM signal from the Arduino to the enable pins of the L293D, which effectively varies the average voltage delivered to each motor and, in turn, its rotational speed and the robot's heading.

1) *PWM Speed Control*: Fig. 14 shows the general form of the PWM waveform applied to each motor-enable pin, where t_{on} is the pulse "on" time within one period t .

The average (DC-equivalent) voltage delivered to the motor is

$$A = \frac{1}{t} \int_0^{t_{on}} V dt = \frac{1}{t} \int_0^{t_{on}} 12 dt = 12 \frac{t_{on}}{t} = 12 \rho, \quad (3)$$

where $\rho = t_{on}/t$ is the duty cycle of the PWM control signal. Equation (3) shows that varying the duty cycle of

TABLE II
EFFECT OF PWM DUTY CYCLE ON MOTOR SPEED

Duty Cycle (ρ)	Voltage Across Motor (V)	Speed (RPM)
0%	0	0
25%	3	16
50%	6	29
75%	9	45
100%	12	58

TABLE III
COMPARISON BETWEEN A STANDARD TACHOMETER AND THE
CONSTRUCTED TACHOMETER

Voltage (V)	RPM (Standard)	RPM (Constructed)	Error (%)
3	70	75	6.25
6	120	132	9.3
9	184	173	6.67
12	258	235	10.4
15	330	284	12.4

the Arduino-generated PWM signal effectively varies the DC voltage supplied to the motors, and hence their speed [9].

2) *Practical Results:* Table II lists the motor speed (RPM, with rotor inertia neglected) measured in the laboratory for several PWM duty cycles.

B. Digital Revolution Counter

A digital RPM (revolutions-per-minute) counter – conceptually similar to a commercial tachometer – was built around the IR break-beam sensor of Fig. 9. A black mark on the wheel interrupts/reflects the IR beam once per revolution, generating a digital pulse that the Arduino counts on a hardware interrupt. In this implementation, the accumulated count is converted to a distance update every 5 s (i.e., 12 times per minute) and shown on the LCD.

The constructed tachometer was benchmarked against a commercial reference instrument across a range of motor voltages; results are summarised in Table III. The percentage error remained within approximately 6–12% over the tested range, which was judged acceptable for the maximum operating speed of interest (approximately 250 RPM).

C. Section Summary

PWM is an effective technique for obtaining an analog-like (continuously variable) result – in this case, motor voltage and hence speed – using purely digital control: a square wave is switched between fully on and fully off, and the fraction of time spent on (the duty cycle) determines the effective average voltage delivered to the load. The constructed tachometer, while less accurate than a commercial reference instrument, remains within an acceptable error margin for the speeds relevant to this platform and is adequate for the distance-display feature.

V. KINEMATICS OF THE ROBOT

A. Differential Steering Model

The platform uses a differential-steering arrangement, familiar from everyday devices such as a wheelchair: two wheels,

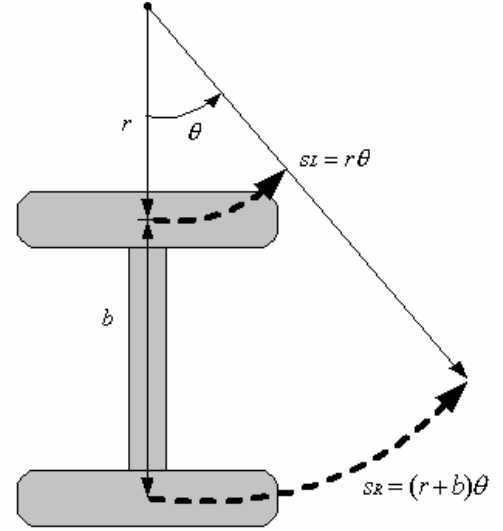


Fig. 15. Differential-steering model: S_L and S_R are the arc lengths travelled by the left and right wheels.

mounted on a common axis, are driven independently, providing both propulsion and steering, while additional passive (caster) wheels provide support. If both drive wheels turn at the same rate, the robot moves in a straight line; if one wheel turns faster than the other, the robot follows a curved path; and if the two wheels turn at equal speed but in opposite directions, the robot pivots in place [7]. The geometry of a general turn is shown in Fig. 15.

For a turn through angle θ with turning radius r (measured to the midpoint of the axle) and wheel separation b , the displacements of the left and right wheels, S_L and S_R , and the displacement of the axle midpoint, S_M , are given by

$$S_L = r\theta, \quad (4)$$

$$S_R = (r + b)\theta, \quad (5)$$

$$S_M = \left(r + \frac{b}{2}\right)\theta. \quad (6)$$

B. Mechanical Design

The chassis is fabricated from acrylic sheet, chosen for its favourable strength-to-weight ratio: it supports the required load while keeping the overall platform light. A detailed orthographic projection of the chassis is provided in the appendix of the original project documentation.

1) *Test Track Design:* The evaluation track (Fig. 16) consists of three U-turns of differing radii. The robot was able to negotiate two of the three U-turns cleanly; on the remaining (tighter) initial U-turn it briefly left the track before re-acquiring and retracing the correct path.

2) *Photographs of the Completed Platform:* Photographs of the assembled robot are shown in Fig. 17 (top view) and Fig. 18 (side view).

C. Turning Radius

The turning radius of a vehicle is the radius of the smallest circular (U-turn) path that it is capable of making. Fig. 19

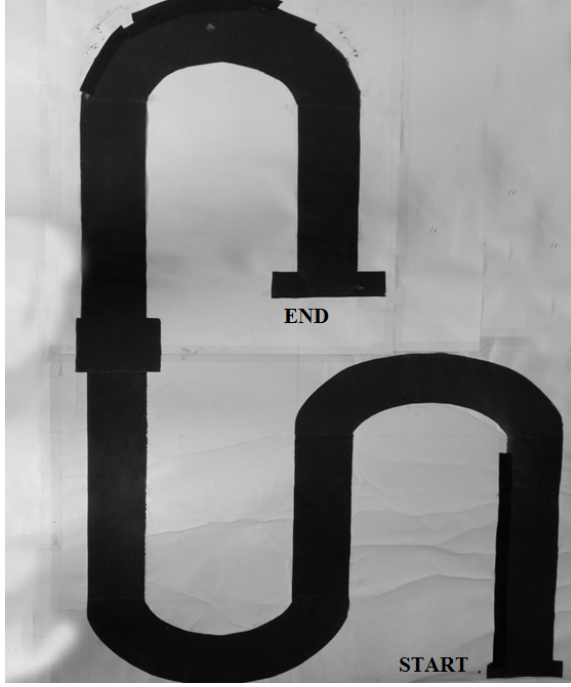


Fig. 16. Test track used to evaluate the line-following robot, consisting of three U-turns of differing radii.

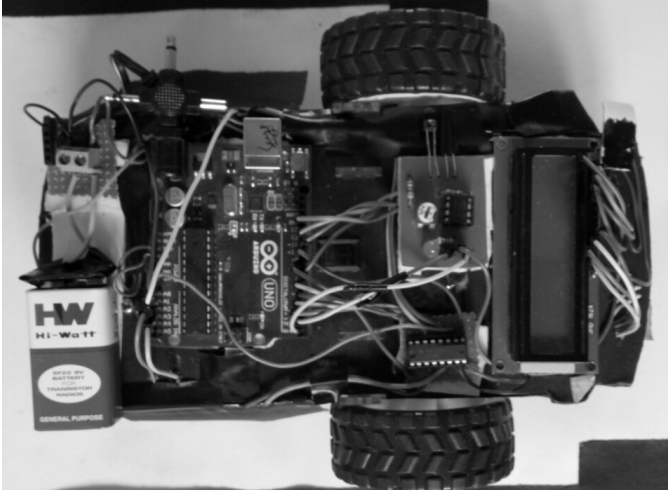


Fig. 17. Top view of the completed line-following robot.

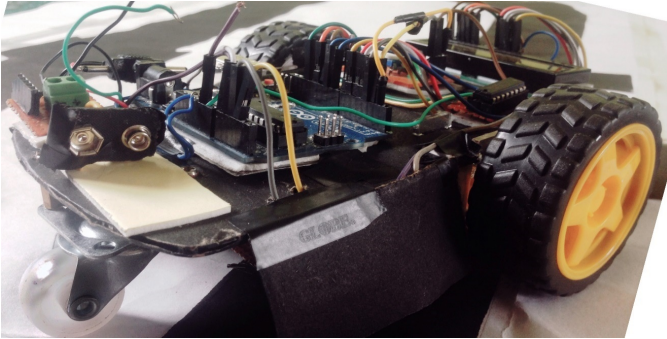


Fig. 18. Side view of the completed line-following robot.

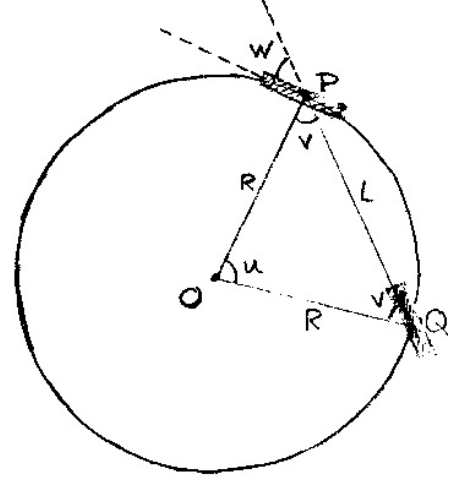


Fig. 19. Geometry used to derive the minimum turning radius.

shows the geometry of the robot mid-turn, with the front and rear wheel centres denoted P and Q , respectively, the centre of the instantaneous turning circle denoted O , and $L = PQ$ the wheelbase length [11]. The sharpest possible turn corresponds to the smallest admissible value of the turning-circle radius R ; since PQ is a chord of that circle, and assuming the front wheel can be steered by at most an angle w relative to straight-ahead, the triangle OPQ satisfies

$$u + 2v = 180^\circ, \quad (7)$$

$$w + v = 90^\circ, \quad (8)$$

$$L^2 = R^2 + R^2 - 2R^2 \cos(u). \quad (9)$$

Combining (7)–(9) gives

$$R = \frac{L}{2 \sin w}. \quad (10)$$

With the total chassis length $L = 172$ mm and taking the maximum value $\sin w = 1$, the minimum turning radius is $R = L/2 = 87.2$ mm.

The velocity at which the robot can safely negotiate a turn of radius r is limited by the available friction between the wheels and the surface. Approximating the manoeuvre as circular motion with gravitational acceleration g and coefficient of friction μ between the wheel material and the track surface,

$$v = \sqrt{\mu r g}. \quad (11)$$

Taking a design turning radius of $r = 100$ mm, $g = 9.81 \times 10^3$ mm/s², and $\mu = 0.07$ (rubber on cardboard) [11],

$$v = \sqrt{0.07 \times 87.5 \times 9810} \approx 245.2 \text{ mm/s} = 24.52 \text{ cm/s}, \quad (12)$$

which corresponds to a wheel speed of approximately 1.4 rev/s (84 RPM). Accordingly, the design turning radius used for the evaluation track was kept no smaller than 100 mm.

D. Cost Estimation

Table IV summarises the bill of materials for the prototype. Excluding labour, the total component cost was approximately NRs. 6,045.

TABLE IV
BILL OF MATERIALS

Item	Qty.	Rate (NRs.)	Total (NRs.)
Arduino Uno	1	2000	2000
Chassis	1	150	150
DC gear motor	2	500	1000
Rear wheel	2	100	200
Front (caster) wheel	1	150	150
LCD	1	450	450
Motor driver (L293D)	1	350	350
Jumper wires (set)	20	10	200
Photodiode	5	30	150
IR LED	5	15	75
LM358 comparator	1	80	80
LM7805 regulator	1	45	45
AAA battery	4	15	60
9 V battery	1	80	80
Battery holder	1	50	50
Switches	1	5	5
Miscellaneous	—	—	1000
Total			6045

E. Section Summary

The mechanical design is governed primarily by the differential-steering mechanism and its associated turning radius. Because the calculated minimum turning radius sets a hard mechanical constraint on the tightest curve the platform can track, any track designed for this robot should avoid U-turns tighter than the derived 100 mm design radius.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper has presented the design, implementation, and evaluation of an autonomous, photodiode-sensor-based line-following robot that tracks a black line on a white surface and adjusts its own position to remain on track. The electromechanical platform, weighing approximately 200 g and measuring $192 \times 100 \times 70 \text{ mm}^3$, achieves a maximum speed of about 180 RPM under no-load, frictionless conditions and a minimum turning radius of 100 mm at a velocity of approximately 24.2 cm/s. The robot reliably re-acquires the line whenever it departs from the expected track. Beyond its immediate function, the project served as an effective vehicle for integrating mechanical design, electronic circuit design, and embedded programming, and it demonstrated the practical potential of a low-cost mechatronic platform built from readily available components.

B. Future Work

Several enhancements were identified during development but could not be completed within the project timeline:

- replacing the binary optical sensors with a dedicated colour sensor for more robust line discrimination;
- incorporating a CCD camera for improved path recognition and more precise trajectory tracking.

APPENDIX A ARDUINO FIRMWARE LISTING

The complete Arduino sketch implementing the control logic of Section III is listed below.

```

1 //=====
2 //===== LINE FOLLOWING ROBOT =====
3 //===== Arduino 1.65 =====
4 #include <LiquidCrystal.h>
5 LiquidCrystal lcd(12, 11, 7, 5, 4, 3, 6);
6
7 float distance;
8 volatile int count = 0; // volatile: modified inside an
9   ISR
10
11 int leftInput1 = A2; // sensor inputs (A0-A3)
12 int leftInput2 = A3;
13 int rightInput1 = A0;
14 int rightInput2 = A1;
15
16 int leftMotor = 9; // PWM outputs to the motor driver
17 int rightMotor = 10;
18
19 int leftValue1 = 0;
20 int rightValue1 = 0;
21 int leftValue2 = 0;
22 int rightValue2 = 0;
23
24 unsigned long lastmillis = 0; // stores the last update
25   timestamp
26
27 void setup()
28 {
29   Serial.begin(9600);
30   lcd.begin(2, 20); // initialise a 2x20 LCD
31   lcd.setCursor(0, 0);
32   lcd.print("distance (cm)");
33
34   pinMode(leftMotor, OUTPUT);
35   pinMode(rightMotor, OUTPUT);
36   attachInterrupt(0, techno, RISING); // INT0 on pin 2,
37     rising edge
38 }
39
40 void loop()
41 {
42   leftValue1 = digitalRead(leftInput1);
43   rightValue1 = digitalRead(rightInput1);
44   leftValue2 = digitalRead(leftInput2);
45   rightValue2 = digitalRead(rightInput2);
46
47   if (millis() - lastmillis >= 6000) // update every ~6
48     s
49   {
50     detachInterrupt(0);
51     distance = distance + 2 * 3.1416 * 3 * count;
52     lcd.setCursor(0, 1);
53     lcd.print(distance);
54     count = 0; // reset the revolution count
55     lastmillis = millis();
56     attachInterrupt(0, techno, RISING);
57   }
58
59   if (leftValue1 == HIGH && leftValue2 == HIGH &&
60       rightValue1 == HIGH && rightValue2 == HIGH)
61   {
62     // all sensors off track: pause briefly, then sweep
63     to search
64     digitalWrite(leftMotor, LOW);
65     digitalWrite(rightMotor, LOW);
66     delay(2000);
67     digitalWrite(leftMotor, LOW);
68     digitalWrite(rightMotor, HIGH);
69     delay(1000);
70   }
71   else if (leftValue1 == HIGH && leftValue2 == LOW &&
72            rightValue1 == LOW && rightValue2 == LOW)
73   {
74     analogWrite(leftMotor, 191);
75     analogWrite(rightMotor, 185); // move right
76   }
77   else if (leftValue1 == HIGH && leftValue2 == HIGH &&
78            rightValue1 == LOW && rightValue2 == LOW)
79   {
80     analogWrite(leftMotor, 191);
81     analogWrite(rightMotor, 64); // turn right
82   }
83   else if (leftValue1 == HIGH && leftValue2 == HIGH &&
84            rightValue1 == HIGH && rightValue2 == LOW)

```

```

81  {
82    analogWrite(leftMotor, 255);
83    analogWrite(rightMotor, 64); // sharp turn right
84  }
85  else if (leftValue1 == LOW && leftValue2 == LOW &&
86          rightValue1 == LOW && rightValue2 == HIGH)
87  {
88    analogWrite(leftMotor, 127);
89    analogWrite(rightMotor, 191); // move left
90  }
91  else if (leftValue1 == LOW && leftValue2 == LOW &&
92          rightValue1 == HIGH && rightValue2 == HIGH)
93  {
94    analogWrite(leftMotor, 64);
95    analogWrite(rightMotor, 191); // turn left
96  }
97  else if (leftValue1 == LOW && leftValue2 == HIGH &&
98          rightValue1 == HIGH && rightValue2 == HIGH)
99  {
100    analogWrite(leftMotor, 64);
101    analogWrite(rightMotor, 255); // sharp turn left
102  }
103  else if (leftValue1 == LOW && leftValue2 == LOW &&
104          rightValue1 == LOW && rightValue2 == LOW)
105  {
106    analogWrite(rightMotor, 255);
107    analogWrite(leftMotor, 255); // go straight
108  }
109 }
110
111 void techno()
112 {
113   count++; // tachometer interrupt service routine
114 }

```

Listing 1. Line-following and distance-measurement firmware (Arduino 1.65).

ACKNOWLEDGMENT

The authors thank Mr. Anand Gachadar, Project Coordinator, and Mr. Bikalpa Upadhayaya, Project Supervisor, both of the Department of Electrical and Electronics Engineering, Kathmandu University, for their guidance throughout this project, and gratefully acknowledge the authors of the references consulted during the work.

REFERENCES

- [1] B. Klaus and P. Horn, *Robot Vision*. Cambridge, MA, USA: MIT Press, 1986.
- [2] J. Warren, J. Adams, and H. Molle, "Arduino for robotics," in *Arduino Robotics*. New York, NY, USA: Apress, 2014, pp. 51–83.
- [3] Jim, "PWM/PID/servo motor control," 2005. [Online]. Available: <http://www.uoxray.uoregon.edu>. Accessed: Dec. 15, 2015.
- [4] Komonya, S. Tachi, and K. Tanie, "A method for autonomous locomotion of mobile robots," *J. Robot. Soc. Jpn.*, vol. 2, pp. 222–231, 1984.
- [5] S. Monk, *Programming Arduino: Getting Started with Sketches*. New Delhi, India: Tata McGraw-Hill, 2012.
- [6] Open Source Community, "Open source sketch," Jan. 2015. [Online]. Available: <https://www.arduino.cc/en/Guide/Introduction>. Accessed: Nov. 25, 2015.
- [7] A. Parsad, "Line following robot," Dept. Electron. Commun. Eng., Visvesvaraya Technol. Univ., Bangalore, India, 2005.
- [8] S. Bhatia, "Engineering garage," May 23, 2011. [Online]. Available: <http://www.engineersgarage.com/tachometer-microcontroller-circuit-project>. Accessed: Mar. 3, 2016.
- [9] S. Debnath and Md. Jinnah, "Digital RPM meter using Arduino," Dept. Elect. Eng., Ahsanullah Univ. Sci. Technol., Dhaka, Bangladesh, 2012.
- [10] "Alldatasheet," Mar. 2003. [Online]. Available: <http://www.alldatasheet.com>. Accessed: Dec. 6, 2015.
- [11] "Turning radius of the car," Wordpress, Jul. 2013. [Online]. Available: <https://goodmaths.wordpress.com/2013/07/19/turning-radius-of-a-car/>. Accessed: Jun. 28, 2016.